

CUDA Fundamentals

1. Overview

Definition

CUDA (Compute Unified Device Architecture) is NVIDIA's platform and programming model for running general-purpose computations on NVIDIA GPUs. It extends languages such as C and C++ with APIs and keywords that let a program:

1. Allocate memory on a GPU.
2. Copy input data between CPU and GPU memory.
3. Launch a GPU function, called a **kernel**, across many parallel threads.
4. Synchronize work and check for errors.
5. Copy the result back and release resources.

CUDA is both:

- A **programming model**: grids, blocks, threads, kernels, memory spaces, and synchronization rules.
- A **runtime API**: functions such as `cudaMalloc`, `cudaMemcpy`, `cudaFree`, and `cudaDeviceSynchronize`.

Why it matters

A modern GPU contains many execution units designed to perform the same kind of operation on large amounts of data. A CPU is optimized for low-latency execution of a few complex instruction streams; a GPU is optimized for high throughput across thousands or millions of similar tasks.

CUDA matters when a problem has substantial **data parallelism**. Examples include applying the same mathematical operation to every pixel, matrix element, simulation particle, or neural-network activation.

Where CUDA is used in real systems

- Deep-learning training and inference
- Scientific simulations and weather modeling
- Image, video, and signal processing
- 3D rendering and ray tracing support workloads
- Financial simulations and risk analysis
- Genomics and bioinformatics
- Database query acceleration
- Search, recommendation, and vector-processing systems

Why interviewers ask about it

CUDA questions reveal whether a candidate can:

- Decompose a problem into parallel work.
- Map data to a hierarchy of threads, blocks, and grids.
- Reason about two processors and separate memory spaces.

- Distinguish execution ordering from synchronization.
- Handle API failures and asynchronous errors correctly.
- Avoid races, out-of-bounds accesses, and invalid barriers.

For placement interviews, interviewers usually value a correct mental model more than memorization of hardware limits.

2. Core Idea

Intuition

Suppose two vectors contain one million numbers and we want to add corresponding elements. A normal CPU loop may process elements one after another:

```
for (int i = 0; i < n; ++i) {
    c[i] = a[i] + b[i];
}
```

Every output element is independent, so CUDA assigns different elements to different GPU threads. Each thread computes its own index and performs one addition.

Real-world analogy

Imagine grading 10,000 independent multiple-choice answer sheets:

- The **host (CPU)** is the coordinator.
- The **device (GPU)** is a large examination hall of workers.
- A **kernel** is the instruction given to every worker: "grade the sheet assigned to you."
- A **thread** is one worker.
- A **block** is one team of workers that can meet and share a local worktable.
- A **grid** is the complete collection of teams assigned to one kernel launch.
- **threadIdx** is a worker's position inside a team.
- **blockIdx** is the team's position in the grid.
- **blockDim** is the number of workers arranged in one team.
- **gridDim** is the number of teams arranged in the grid.

Workers in one team can coordinate with `__syncthreads()`. Ordinary workers in different teams cannot use it to meet at a common barrier.

Small end-to-end example: vector addition

```
#include <cuda_runtime.h>
#include <cstdlib>
#include <iostream>

#define CUDA_CHECK(call)
do {
```

```
\
\
```

```

        cudaError_t error = (call);
        if (error != cudaSuccess) {
            std::cerr << "CUDA error at " << __FILE__ << ':' << __LINE__
                << ": " << cudaGetErrorString(error) << '\n';
            std::exit(EXIT_FAILURE);
        }
    } while (0)

__global__ void vectorAdd(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    constexpr int n = 1000;
    constexpr std::size_t bytes = n * sizeof(float);

    float h_a[n], h_b[n], h_c[n];
    for (int i = 0; i < n; ++i) {
        h_a[i] = static_cast<float>(i);
        h_b[i] = 2.0f * i;
    }

    float *d_a = nullptr, *d_b = nullptr, *d_c = nullptr;
    CUDA_CHECK(cudaMalloc(&d_a, bytes));
    CUDA_CHECK(cudaMalloc(&d_b, bytes));
    CUDA_CHECK(cudaMalloc(&d_c, bytes));

    CUDA_CHECK(cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice));

    constexpr int threadsPerBlock = 256;
    const int blocks = (n + threadsPerBlock - 1) / threadsPerBlock;
    vectorAdd<<<blocks, threadsPerBlock>>>>(d_a, d_b, d_c, n);

    CUDA_CHECK(cudaGetLastError()); // Detect launch-configuration errors.
    CUDA_CHECK(cudaDeviceSynchronize()); // Detect errors during execution.

    CUDA_CHECK(cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost));

    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));

    std::cout << "h_c[10] = " << h_c[10] << '\n'; // Expected: 30
}

```

Step-by-step execution

1. CPU code creates and initializes host arrays.

2. `cudaMalloc` reserves three buffers in device memory.
3. `cudaMemcpy` transfers the two inputs from host to device.
4. The host calculates enough blocks using ceiling division.
5. `vectorAdd<<<blocks, threadsPerBlock>>>` places GPU work into a CUDA stream.
6. Every GPU thread computes a unique global index.
7. The bounds check disables extra threads in the last partially used block.
8. Error checks distinguish launch failure from execution failure.
9. Results are copied to host memory.
10. `cudaFree` releases device allocations.

3. Important Subtopics

3.1 CUDA programming model

What it means

CUDA exposes parallel execution as a hierarchy:

```
Host program
  |
  +-- launches Kernel
        |
        +-- Grid
              |
              +-- Block 0 -- Thread 0, Thread 1, ...
              +-- Block 1 -- Thread 0, Thread 1, ...
              +-- ...
```

A kernel launch creates one **grid**. A grid contains **blocks**, and each block contains **threads**. All threads execute the same kernel code, but built-in indices let them select different data.

The model is often described as **SIMT**: Single Instruction, Multiple Threads. Threads are written as independent logical threads, while hardware schedules groups called **warps**. On current CUDA GPUs, a warp contains 32 threads. If threads in a warp take different branches, the paths may have to execute separately, reducing efficiency.

Why it matters

The hierarchy provides scalability. The program describes many blocks without depending on the exact number of GPU cores. The GPU scheduler assigns ready blocks to available streaming multiprocessors (SMs).

Example

For 1,000 elements and 256 threads per block:

```
blocks = ceil(1000 / 256) = 4
launched threads = 4 * 256 = 1024
```

```
useful threads = 1000
extra threads = 24, rejected by `if (i < n)`
```

Common interview angle

Explain why blocks must be independently executable. A normal kernel must not assume that block 0 runs before block 1, or that all blocks are simultaneously resident. This lets the same grid scale across GPUs with different resources.

3.2 `__global__`, `__device__`, and `__host__`

These are CUDA function execution-space specifiers. The terms **host** and **device** also describe the CPU side and GPU side of the system.

Specifier	Called from	Executes on	Return rule	Typical use
<code>__global__</code>	Host; device with dynamic parallelism	Device	Must return <code>void</code>	Kernel entry point
<code>__device__</code>	Device	Device	Any valid return type	GPU helper function
<code>__host__</code>	Host	Host	Normal C++ rules	Explicit CPU function
<code>__host__</code> <code>__device__</code>	Host or device	Compiled for both	Normal rules subject to each target	Small shared utility

`__global__`

A `__global__` function is a **kernel**. It runs on the device and is normally launched by host code with `<<<...>>>` syntax.

```
__global__ void square(float* values, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) values[i] *= values[i];
}

square<<<numBlocks, threadsPerBlock>>>(d_values, n);
```

Important facts:

- A kernel is normally asynchronous with respect to the host.
- A kernel returns `void`; results are written through memory.
- Every launched thread begins at the same kernel entry point.
- The launch configuration chooses the grid and block dimensions.

`__device__`

A `__device__` function runs on the GPU and is called by GPU code.

```
__device__ float clampZero(float x) {
    return x < 0.0f ? 0.0f : x;
}

__global__ void relu(float* values, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) values[i] = clampZero(values[i]);
}
```

It is useful for reusable device-side logic. It is not directly called like a normal function from host code.

`__host__`

A `__host__` function runs on the CPU. Ordinary C++ functions are host functions by default, so the keyword is often omitted.

```
__host__ int blocksFor(int n, int blockSize) {
    return (n + blockSize - 1) / blockSize;
}
```

Combining `__host__ __device__` asks the compiler to create CPU and GPU versions. Both compilations must accept the function body; calling host-only APIs such as `std::cout` from the device version is generally invalid.

Common interview angle

Interviewers often ask where each function executes and who can call it. Avoid saying `__global__` means "global memory"; it describes a kernel function, not a memory location.

3.3 Kernel launches

Syntax

```
kernel<<<grid, block, sharedMemoryBytes, stream>>>(arguments...);
```

Only the first two execution-configuration arguments are required:

- `grid`: number and shape of blocks (`dim3`).
- `block`: number and shape of threads per block (`dim3`).
- `sharedMemoryBytes`: optional dynamic shared memory per block; default is 0.
- `stream`: optional CUDA stream; default is the default stream.

Examples:

```
// One-dimensional launch
process<<<80, 256>>>(data, n);

// Two-dimensional launch for an image
dim3 block(16, 16);
dim3 grid((width + block.x - 1) / block.x,
          (height + block.y - 1) / block.y);
processImage<<<grid, block>>>(pixels, width, height);
```

Why launches are usually asynchronous

After enqueueing a kernel, the CPU can continue without waiting for GPU completion. This enables overlap of CPU work, GPU work, and transfers when streams and suitable memory are used. It also means some errors appear only at a later synchronization point.

Operations in the same stream execute in issue order. Different streams may overlap when their dependencies and hardware permit it.

Choosing launch dimensions

For a one-dimensional array, a common starting point is 128 or 256 threads per block. The correct choice depends on register use, shared memory, occupancy, memory behavior, and measurements. Never omit the bounds check merely because a test input happens to be divisible by the block size.

Common interview angle

Candidates are often asked to calculate the grid size and explain the fourth launch argument. The expected ceiling-division formula is:

```
int blocks = (n + threadsPerBlock - 1) / threadsPerBlock;
```

3.4 threadIdx

`threadIdx` is a built-in variable available inside device code. Its type is `dim3`, with components `.x`, `.y`, and `.z`. It identifies a thread **within its block**.

```
int localX = threadIdx.x;
int localY = threadIdx.y;
```

Its values begin at zero and are bounded by the corresponding block dimension:

```
0 <= threadIdx.x < blockDim.x
0 <= threadIdx.y < blockDim.y
0 <= threadIdx.z < blockDim.z
```

`threadIdx.x` is not globally unique. Every block contains a thread whose local index is zero.

Example and interview angle

For `kernel<<<3, 4>>>`, each of the three blocks has local thread indices 0, 1, 2, and 3. To access a unique array element, combine `threadIdx.x` with `blockIdx.x` and `blockDim.x`.

3.5 `blockIdx`

`blockIdx` is a built-in `dim3` value identifying the current block within the grid. Its components also begin at zero:

```
0 <= blockIdx.x < gridDim.x
```

For one-dimensional data, the common global index is:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

Why it matters

`blockIdx` lets different blocks process different tiles or chunks of data. Blocks should normally be written so any block can execute in any order.

Common interview angle

An interviewer may ask whether `blockIdx.x` identifies a thread. It does not: it identifies a block, so all threads in that block see the same `blockIdx`.

3.6 `blockDim`

`blockDim` is a built-in `dim3` value containing the dimensions of the current block. It equals the block configuration supplied at launch.

```
kernel<<<gridSize, 256>>>(...); // Inside: blockDim.x == 256
```

Total threads in a three-dimensional block are:

```
blockDim.x * blockDim.y * blockDim.z
```

Why it matters

It makes indexing code independent of a hard-coded block size. If the launch changes from 128 to 256 threads, the formula still works.

Common interview angle

Do not confuse `blockDim.x` with the number of blocks. `blockDim` describes threads per block; `gridDim` describes blocks per grid.

3.7 `gridDim`

`gridDim` is a built-in `dim3` value containing the number of blocks in each grid dimension for the current kernel launch.

```
kernel<<<10, 256>>>(...); // Inside: gridDim.x == 10
```

It is useful in **grid-stride loops**, where each thread handles multiple elements:

```
__global__ void scale(float* data, int n, float factor) {  
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;  
         i < n;  
         i += blockDim.x * gridDim.x) {  
        data[i] *= factor;  
    }  
}
```

The stride equals the total number of threads in the one-dimensional grid. Grid-stride loops allow a fixed-size grid to process arbitrarily large arrays.

Common interview angle

Why use a grid-stride loop instead of one thread per element? It can limit launch size, reuse threads, support large inputs, and make launch tuning easier while retaining coalesced access within each iteration.

3.8 Multi-dimensional indexing

Images and matrices naturally use two dimensions:

```
__global__ void brighten(unsigned char* image, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < width && y < height) {
        int linearIndex = y * width + x;
        image[linearIndex] = min(255, image[linearIndex] + 20);
    }
}
```

Launch:

```
dim3 block(16, 16); // 256 threads per block
dim3 grid((width + block.x - 1) / block.x,
          (height + block.y - 1) / block.y);
brighten<<<grid, block>>>(d_image, width, height);
```

The logical dimensions improve readability, but row-major memory is still linear: $(\text{row} * \text{width} + \text{column})$.

3.9 cudaMalloc

What it means

`cudaMalloc` allocates linear memory accessible by the device:

```
cudaError_t cudaMalloc(void** devPtr, size_t size);
```

Modern C++ CUDA headers allow common typed-pointer usage such as:

```
float* d_data = nullptr;
cudaError_t status = cudaMalloc(&d_data, n * sizeof(float));
```

Important rules

- The size is in **bytes**, not element count.
- It writes the allocated address into the pointer supplied by the caller.
- It does not initialize the memory.
- The allocation should be checked for failure.
- The allocation must eventually be released with `cudaFree`.
- Device memory is limited; allocation can fail with an out-of-memory error.

Common interview angle

`cudaMalloc` resembles `malloc` in purpose, but the pointer refers to device-accessible memory and the API reports errors with `cudaError_t`. Do not dereference a traditional device pointer directly in ordinary host code.

3.10 `cudaMemcpy`

What it means

`cudaMemcpy` copies bytes between memory locations:

```
cudaError_t cudaMemcpy(void* dst, const void* src,
                      size_t count, cudaMemcpyKind kind);
```

Common directions:

Copy kind	Meaning
<code>cudaMemcpyHostToDevice</code>	CPU memory to GPU memory
<code>cudaMemcpyDeviceToHost</code>	GPU memory to CPU memory
<code>cudaMemcpyDeviceToDevice</code>	One device allocation to another
<code>cudaMemcpyHostToHost</code>	Host memory to host memory
<code>cudaMemcpyDefault</code>	Infer direction when supported by pointer attributes/unified virtual addressing

Example:

```
CUDA_CHECK(cudaMemcpy(d_data, h_data, bytes, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(h_result, d_result, bytes, cudaMemcpyDeviceToHost));
```

Why it matters

PCIe or other interconnect transfers can be expensive compared with arithmetic. Efficient applications reduce unnecessary transfers, batch them, and sometimes overlap transfers with computation using `cudaMemcpyAsync`, streams, and pinned host memory.

Common interview angle

The order is **destination first, source second**, like `memcpy`. Common errors are reversing pointers, using the wrong direction, or copying `n` bytes instead of `n * sizeof(T)` bytes.

3.11 `cudaFree`

`cudaFree(pointer)` releases memory previously allocated by APIs such as `cudaMalloc`.

```
CUDA_CHECK(cudaFree(d_data));  
d_data = nullptr; // Prevent accidental reuse in host code.
```

Why it matters

Failing to free repeatedly allocated buffers causes device-memory leaks. A long-running service can eventually fail future allocations even if each individual request is small.

`cudaFree(nullptr)` is safe. Freeing an invalid pointer, freeing the same pointer twice, or using a pointer after it has been freed is a bug.

Common interview angle

Process termination lets the CUDA runtime reclaim resources, but explicit cleanup is still necessary for reusable libraries and long-running applications. Also be aware that some CUDA API calls may synchronize or expose earlier asynchronous errors; do not use `cudaFree` as the deliberate correctness barrier for a program.

3.12 Error handling

CUDA runtime calls return `cudaError_t`. Successful calls return `cudaSuccess`. Convert an error to readable text using `cudaGetErrorString`.

Two error stages for a kernel

```
kernel<<<grid, block>>>(...);  
CUDA_CHECK(cudaGetLastError());  
CUDA_CHECK(cudaDeviceSynchronize());
```

1. **Launch errors:** invalid configuration, invalid arguments, or similar problems detectable when the launch is issued. `cudaGetLastError()` reports and resets the thread-local last error.
2. **Execution errors:** illegal memory access and other failures that occur while the asynchronous kernel runs. A synchronization call such as `cudaDeviceSynchronize()` can report them.

During normal optimized execution, an application need not synchronize after every kernel if later stream operations provide the needed ordering. During development, explicit synchronization makes failures easier to associate with the correct kernel.

Reusable checking pattern

```
#define CUDA_CHECK(call)  
do {  
    cudaError_t error = (call);  
    \
```

```

        if (error != cudaSuccess) {
            std::cerr << cudaGetErrorString(error) << '\n';
            std::exit(EXIT_FAILURE);
        }
    } while (0)

```

Production libraries often return or throw an error rather than terminating the entire process. The correct policy depends on ownership of the application.

Common interview angle

Checking only `cudaGetLastError()` is insufficient because it may confirm that a launch was accepted even though execution later performs an illegal access.

3.13 Synchronization with `__syncthreads()`

What it means

`__syncthreads()` is a barrier for threads in the **same block**. A thread reaching it waits until all participating threads in that block reach the same barrier. Memory accesses made before the barrier become visible to the block's threads after the barrier, which is especially important when sharing data through shared memory.

Example: safely sharing a block tile

```

__global__ void addBlockSum(const float* input, float* output) {
    __shared__ float tile[256];
    int global = blockIdx.x * blockDim.x + threadIdx.x;

    tile[threadIdx.x] = input[global];
    __syncthreads();

    // After the barrier, every thread may read values written by its block.
    float firstValueInBlock = tile[0];
    output[global] = tile[threadIdx.x] + firstValueInBlock;
}

```

Without the barrier, one thread may read `tile[0]` before thread 0 writes it.

Critical restriction: barrier convergence

All threads in a block must reach the barrier in a compatible way. This is unsafe when only some threads enter the branch:

```
if (threadIdx.x < 16) {  
    __syncthreads(); // Wrong if the rest of the block skips it.  
}
```

A barrier is allowed in conditional code only when the condition is uniform across the entire block, so either every thread reaches it or none does.

What it does not do

- It does not synchronize different blocks.
- It does not synchronize the host with the device.
- It does not replace atomic operations when multiple threads update the same location.
- It does not make divergent algorithms safe by itself.

Common interview angle

Distinguish three concepts:

- `__syncthreads()`: device-side, one block.
- `cudaDeviceSynchronize()`: host waits until preceding device work is complete.
- Atomics: serialize updates to a shared memory location to avoid lost updates.

4. Real-World Example

GPU image brightness adjustment

A photo editor must increase the brightness of a 4K grayscale image. A 3840×2160 image contains 8,294,400 pixels. Each pixel can be processed independently.

```
__global__ void adjustBrightness(unsigned char* pixels,  
                                int width,  
                                int height,  
                                int amount) {  
    int x = blockIdx.x * blockDim.x + threadIdx.x;  
    int y = blockIdx.y * blockDim.y + threadIdx.y;  
  
    if (x >= width || y >= height) return;  
  
    int i = y * width + x;  
    int adjusted = static_cast<int>(pixels[i]) + amount;  
    pixels[i] = static_cast<unsigned char>(min(255, max(0, adjusted)));  
}
```

Host-side flow:

```

std::size_t bytes = static_cast<std::size_t>(width) * height;
unsigned char* d_pixels = nullptr;

CUDA_CHECK(cudaMalloc(&d_pixels, bytes));
CUDA_CHECK(cudaMemcpy(d_pixels, h_pixels, bytes, cudaMemcpyHostToDevice));

dim3 block(16, 16);
dim3 grid((width + block.x - 1) / block.x,
          (height + block.y - 1) / block.y);
adjustBrightness<<<grid, block>>>(d_pixels, width, height, 20);

CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaMemcpy(h_pixels, d_pixels, bytes, cudaMemcpyDeviceToHost));
CUDA_CHECK(cudaFree(d_pixels));

```

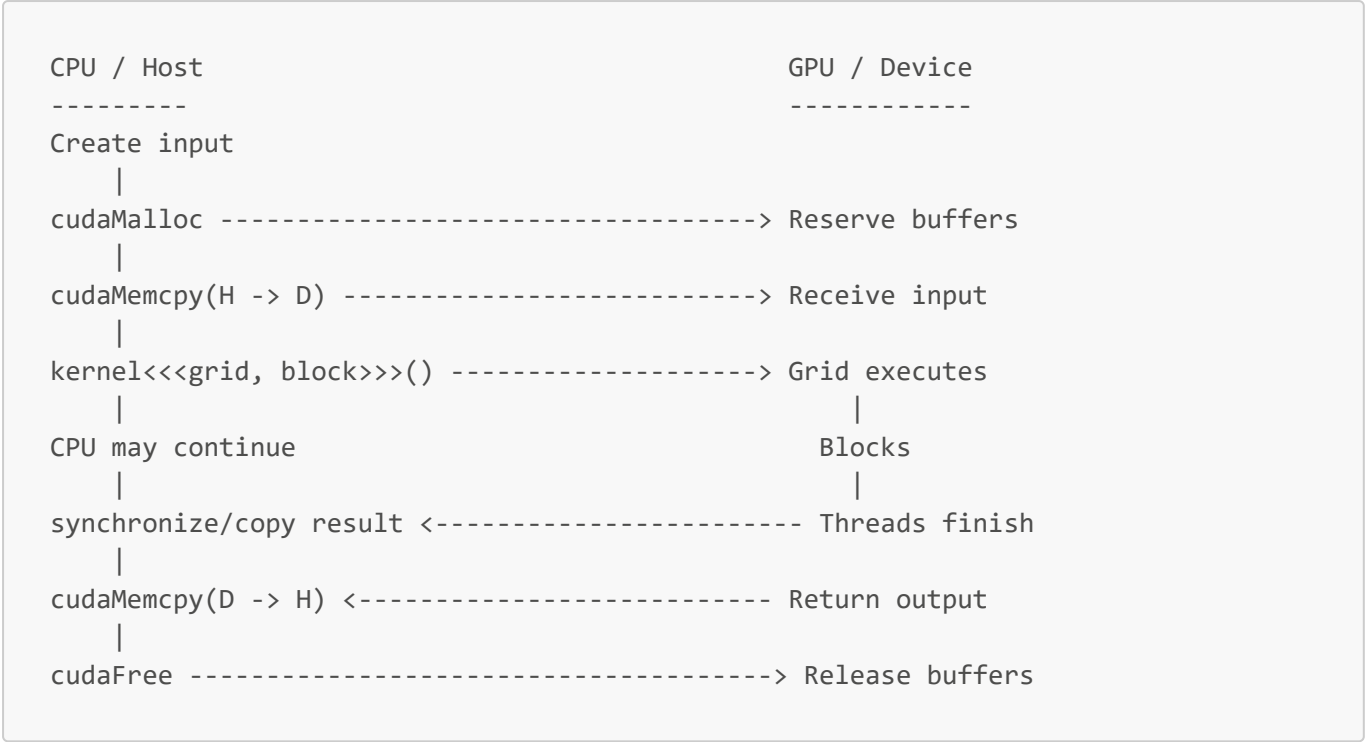
How the requested concepts appear

Concept	Role in the image pipeline
Host	Loads image, allocates memory, launches work, saves output
Device	Executes brightness calculation across pixels
<code>__global__</code>	Marks <code>adjustBrightness</code> as a kernel
Kernel launch	Creates a 2D grid of 16 × 16 thread blocks
<code>threadIdx</code>	Pixel position within a tile
<code>blockIdx</code>	Tile position within the image
<code>blockDim</code>	Tile width and height
<code>gridDim</code>	Total tile layout for the image
<code>cudaMalloc</code>	Allocates the GPU image buffer
<code>cudaMemcpy</code>	Moves pixels to and from the GPU
<code>cudaFree</code>	Releases the GPU buffer
Error handling	Detects allocation, transfer, launch, and execution errors
<code>__syncthreads()</code>	Not needed here because pixels are independent; useful for filters that share neighboring pixels

For a blur filter, threads would often load a tile plus neighboring pixels into shared memory, call `__syncthreads()`, and then read the tile safely to compute the filter.

5. Diagrams / Mental Models

5.1 Host-device workflow



5.2 One-dimensional indexing

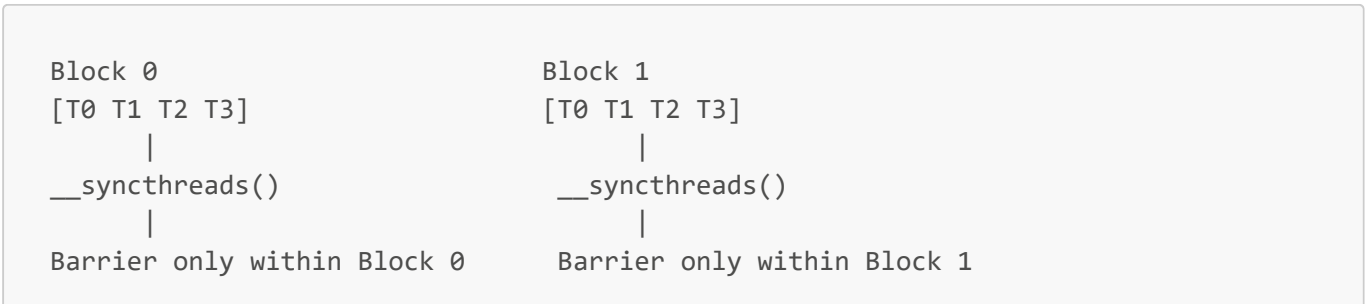
For `kernel<<<3, 4>>>`:



5.3 Dimension reference

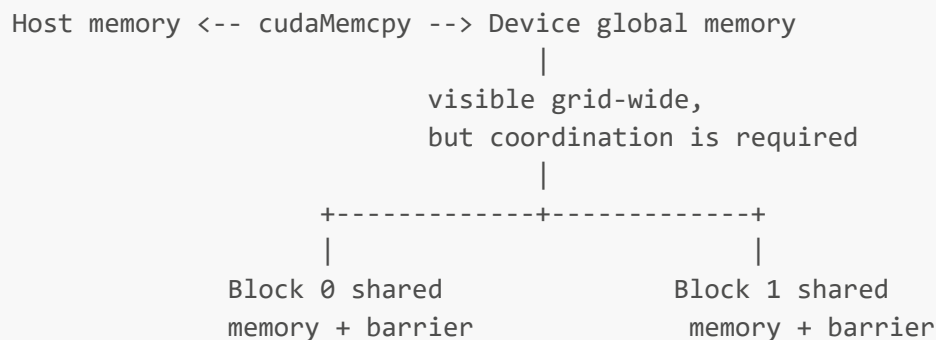
Built-in	Question it answers	Same for which threads?
<code>threadIdx</code>	Where am I inside my block?	Different threads usually differ
<code>blockIdx</code>	Which block am I in?	Same for all threads in one block
<code>blockDim</code>	How many threads is my block shaped with?	Same for all threads in the grid
<code>gridDim</code>	How many blocks is my grid shaped with?	Same for all threads in the grid

5.4 Synchronization scope



There is no cross-block meeting at this barrier.

5.5 Memory and ordering mental model



Shared memory belongs to a block. Global memory is accessible more broadly, but access scope alone does not prevent races or create a grid-wide barrier.

6. Common Interview Questions

Q1. What is CUDA, and how is it different from ordinary C++?

Answer: CUDA is NVIDIA's parallel-computing platform and programming model. CUDA C++ adds kernel syntax, execution-space specifiers, built-in thread indices, memory APIs, and synchronization mechanisms to C++. Host code runs on the CPU; kernels run across many GPU threads.

Interviewer expects: Programming model, host/device distinction, data-parallel purpose.

Common mistake: Describing CUDA as only a C++ library or as a GPU itself.

Q2. What is the difference between a thread, block, and grid?

Answer: A thread is one logical execution of a kernel. Threads are grouped into blocks, whose threads can cooperate through shared memory and block-level synchronization. All blocks created by one kernel launch form a grid. Blocks are scheduled independently.

Interviewer expects: Hierarchy and cooperation scope.

Common mistake: Claiming `__syncthreads()` synchronizes the whole grid.

Q3. Explain `__global__`, `__device__`, and `__host__`.

Answer: `__global__` marks a kernel that executes on the device and is launched with `<<<...>>>`; it returns `void`. `__device__` marks a device function called from device code. `__host__` marks a CPU function and is the default for ordinary functions. `__host__ __device__` can generate both versions when its code is valid for both targets.

Interviewer expects: Caller, execution location, kernel return restriction.

Common mistake: Associating `__global__` with global memory rather than a kernel.

Q4. How do you compute a one-dimensional global thread index?

Answer:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

`blockIdx.x * blockDim.x` is the block's starting offset, and `threadIdx.x` is the position inside that block.

Interviewer expects: Formula and bounds check.

Common mistake: Using only `threadIdx.x`, which repeats in every block.

Q5. Why is a bounds check needed in most kernels?

Answer: Grid sizes are normally rounded up so there are enough complete blocks. The last block may contain threads whose indices exceed the input size. `if (i < n)` prevents out-of-bounds memory access.

Interviewer expects: Ceiling division and partial final block.

Common mistake: Assuming extra threads are automatically disabled.

Q6. What do the arguments inside `<<<...>>>` mean?

Answer: They are `<<<grid dimensions, block dimensions, dynamic shared-memory bytes, stream>>>`. The last two are optional and default to zero shared bytes and the default stream.

Interviewer expects: All four configuration fields and the distinction from normal kernel parameters.

Common mistake: Saying the first value is total threads and the second is number of blocks.

Q7. Is a kernel launch synchronous or asynchronous?

Answer: A kernel launch is normally asynchronous with respect to the host: it enqueues work and may return before the device finishes. Work within the same CUDA stream is ordered. The host can wait using an appropriate synchronization operation, or a later blocking operation may wait as part of its behavior.

Interviewer expects: Host asynchrony plus stream ordering.

Common mistake: Saying all CUDA activity is asynchronous without qualification.

Q8. What is the correct lifecycle of a device buffer?

Answer: Declare a device pointer, allocate bytes with `cudaMalloc`, copy or initialize data as needed, pass the pointer to kernels, copy results if required, and release it using `cudaFree`. Check every CUDA API result.

Interviewer expects: Allocate, transfer, use, transfer back, free.

Common mistake: Dereferencing a conventional device pointer in host code.

Q9. How does `cudaMemcpy` know the direction of a copy?

Answer: The fourth argument specifies a `cudaMemcpyKind`, such as `cudaMemcpyHostToDevice` or `cudaMemcpyDeviceToHost`. With appropriate unified virtual addressing, `cudaMemcpyDefault` may infer it from pointer attributes.

Interviewer expects: Destination-first signature and direction enum.

Common mistake: Reversing source and destination or confusing the kind.

Q10. Why are both `cudaGetLastError()` and `cudaDeviceSynchronize()` used after a kernel during debugging?

Answer: `cudaGetLastError()` detects immediate launch errors. Because execution is asynchronous, runtime failures such as illegal memory access may only be returned when the host synchronizes. The two checks cover different stages.

Interviewer expects: Launch-time versus execution-time errors.

Common mistake: Believing a successful launch check proves the kernel completed correctly.

Q11. What exactly does `__syncthreads()` synchronize?

Answer: It is a barrier among threads in the same block. Threads wait until the block's participating threads reach the barrier, and earlier memory accesses become visible to threads in that block after it. It does not synchronize blocks or the host.

Interviewer expects: Block scope, barrier behavior, memory visibility.

Common mistake: Calling it a grid-wide or CPU-GPU synchronization operation.

Q12. What happens if `__syncthreads()` is inside a branch taken by only some threads?

Answer: The kernel can hang or behave unpredictably because some threads wait at a barrier that other threads never reach. Conditional barriers are safe only when the condition evaluates uniformly for the entire block.

Interviewer expects: Barrier convergence requirement.

Common mistake: Thinking inactive threads are automatically ignored by the barrier.

Q13. What are `blockDim` and `gridDim`?

Answer: `blockDim` is the three-dimensional shape of threads in a block. `gridDim` is the three-dimensional shape of blocks in the grid. They reflect the launch configuration and are available inside the kernel.

Interviewer expects: Correct units: threads versus blocks.

Common mistake: Swapping the two.

Q14. What is a grid-stride loop?

Answer: It is a loop where a thread starts at its global index and increments by the total number of threads in the grid:

```
for (int i = blockIdx.x * blockDim.x + threadIdx.x;
     i < n;
     i += blockDim.x * gridDim.x) {
    // process i
}
```

It lets a limited grid process a large input while maintaining adjacent accesses across neighboring threads.

Interviewer expects: Start, stride formula, purpose.

Common mistake: Incrementing only by `blockDim.x`, causing blocks to overlap work.

Q15. Does `__syncthreads()` prevent all data races?

Answer: No. A barrier controls phase ordering within a block, but if multiple threads perform conflicting non-atomic writes to the same location in the same phase, the race remains. Atomics, ownership rules, or algorithm redesign may be needed.

Interviewer expects: Barrier versus mutual exclusion/atomicity distinction.

Common mistake: Treating a barrier like a mutex.

7. Deep-Dive Questions

1. Why must CUDA blocks be independently executable?

The hardware may run blocks in any order, concurrently or sequentially, based on available registers, shared memory, and execution slots. Independence lets a grid scale to GPUs with different SM counts. If block A waits for a value from block B inside a normal kernel, B might not yet be scheduled, potentially causing deadlock.

Normal solutions include splitting the computation into multiple kernels, because the completion of one kernel before a dependent kernel in the same stream creates a grid-wide phase boundary. Specialized cooperative launches can support grid-wide synchronization, but they impose launch and residency constraints and are not the default answer.

2. How are logical threads mapped to hardware?

Threads in a block are partitioned into warps of 32 threads. An SM schedules warps from resident blocks. Several blocks may reside on one SM if registers, shared memory, thread limits, and block limits allow it. The source model gives each thread its own logical state, while SIMT hardware executes ready warp instructions.

The mapping explains why block size affects occupancy and why branch divergence within a warp can reduce throughput. High occupancy is useful for hiding latency but is not automatically equal to high performance.

3. What guarantees memory visibility around `__syncthreads()`?

For threads in a block, the barrier ensures that memory accesses performed before the barrier are visible to those threads after it, for the relevant shared and global memory operations. This enables staged algorithms: load a shared tile, synchronize, compute using the tile, synchronize again before overwriting it.

It does not create a grid-wide ordering guarantee. For communication beyond a block, use an appropriate kernel boundary, atomics/fences with a correct protocol, or supported cooperative synchronization.

4. How would you process data larger than the number of launched threads?

Use a grid-stride loop. Each thread processes indices separated by `blockDim.x * gridDim.x`. This preserves a simple mapping, lets the program cap the number of blocks, and avoids relying on an extremely large launch. Use a sufficiently wide integer type such as `std::size_t` when indexing arrays that could exceed the range of `int`.

5. Why can an error be reported by a CUDA call that appears unrelated to the faulty kernel?

Kernel execution is asynchronous. A kernel may fail after its launch API has returned successfully. A later synchronizing CUDA call can observe and report that pending error. Therefore, place launch checks and deliberate synchronization near kernels while debugging. In production, maintain correct stream dependencies and attach error reporting to known synchronization points so the failing work can still be identified.

8. Comparison Tables

8.1 Host, device function, and kernel

Property	Host function	Device function	Kernel
Typical declaration	Ordinary C++ / <code>__host__</code>	<code>__device__</code>	<code>__global__</code>
Executes on	CPU	GPU	GPU
Normally called by	Host code	Device code	Host code with <code><<<...>>></code>
Parallel invocation	Normal CPU call	Per calling GPU thread	Many GPU threads from launch
Return value	Normal C++	Normal supported type	<code>void</code>
Has <code>threadIdx</code>	No	Yes when called in device execution	Yes

8.2 Built-in indexing variables

Variable	Represents	Unit	Example use
<code>threadIdx.x</code>	Local position in block	Threads	Select item within a tile
<code>blockIdx.x</code>	Block position in grid	Blocks	Select a tile

Variable	Represents	Unit	Example use
<code>blockDim.x</code>	Block extent	Threads per block	Compute block offset or stride
<code>gridDim.x</code>	Grid extent	Blocks per grid	Compute total grid stride

8.3 CUDA allocation and transfer operations

Operation	Purpose	Important detail
<code>cudaMalloc(&p, bytes)</code>	Allocate device memory	Uninitialized; size is bytes
<code>cudaMemcpy(dst, src, bytes, kind)</code>	Copy memory	Destination comes first
<code>cudaMemset(p, value, bytes)</code>	Set device bytes	Byte-wise value, not arbitrary typed value
<code>cudaFree(p)</code>	Release device memory	Pointer becomes invalid after success

8.4 Synchronization mechanisms

Mechanism	Called from	Scope	Main purpose
<code>__syncthreads()</code>	Device code	One block	Block barrier and memory visibility
<code>cudaDeviceSynchronize()</code>	Host code	Device work issued before it	Host waits; surfaces asynchronous errors
Stream ordering	Host enqueues work	One stream	Order dependent operations without host blocking
Atomic operation	Device code	Target memory operation	Indivisible read-modify-write
Separate kernel launch in same stream	Host code	Grid-wide phase ordering	Finish one kernel before dependent next kernel

8.5 CPU and GPU execution focus

Aspect	CPU	GPU
Optimization goal	Low latency	High throughput
Typical parallelism	Fewer complex threads	Many lightweight threads
Control flow	Handles irregular flow well	Best with similar work across threads
Common CUDA role	Coordinate, allocate, launch	Perform data-parallel computation

Aspect	CPU	GPU
Best-fit workload	Sequential/branch-heavy tasks	Large regular parallel workloads

8.6 `cudaMemcpy` versus `cudaMemcpyAsync`

Aspect	<code>cudaMemcpy</code>	<code>cudaMemcpyAsync</code>
Host behavior	Commonly blocking for relevant transfer semantics	Enqueues copy in a stream
Stream argument	No explicit stream	Explicit stream supported
Potential overlap	Limited	Can overlap with work when requirements are met
Host memory for reliable async transfer	Ordinary pageable memory may be accepted	Pinned host memory is normally required for true asynchronous host transfer
Best starting use	Simple correct programs	Pipelined performance-sensitive programs

9. Common Mistakes

1. **Using only `threadIdx.x` as an array index.** It repeats in every block. Use the global-index formula.
2. **Swapping grid and block launch arguments.** `<<<blocks, threadsPerBlock>>>` is the usual 1D order.
3. **Forgetting the bounds check.** Rounded-up launches create extra threads.
4. **Passing element count instead of byte count.** Allocate and copy `n * sizeof(T)` bytes.
5. **Reversing `cudaMemcpy` arguments.** The order is destination, source, count, direction.
6. **Using the wrong copy direction.** The enum must match the actual pointer locations.
7. **Assuming `cudaMalloc` initializes memory.** It does not; initialize or overwrite before reading.
8. **Ignoring API return values.** Failures then appear as confusing later behavior.
9. **Checking only kernel launch errors.** Execution errors require a later synchronization/error-reporting point.
10. **Synchronizing after every kernel in optimized code.** It can destroy useful overlap; retain only required dependency points after debugging.
11. **Calling `__syncthreads()` in a non-uniform branch.** Some threads can wait forever.
12. **Using `__syncthreads()` for cross-block coordination.** Its scope is one block.
13. **Treating `__syncthreads()` as an atomic operation.** It does not prevent same-phase conflicting writes.
14. **Assuming block execution order.** The scheduler may run blocks in any order.
15. **Hard-coding 256 in the global index.** Use `blockDim.x` so launch tuning remains correct.
16. **Choosing a huge block without checking limits/resources.** Thread, register, and shared-memory limits constrain residency.
17. **Freeing memory too early or using it after free.** Respect stream completion and allocation lifetime.
18. **Expecting every GPU problem to be faster.** Transfer cost, low parallelism, divergence, and small input sizes can make a CPU faster.

10. Edge Cases / Special Cases

Empty input

If `n == 0`, the ceiling formula produces zero blocks. A zero-sized grid launch is invalid. Skip allocation/launch work for an empty dataset or handle it explicitly.

Integer overflow in size calculations

This can overflow before assignment if multiplication is performed using a narrow type:

```
std::size_t bytes = static_cast<std::size_t>(n) * sizeof(float);
```

For very large arrays, use `std::size_t` or another appropriate wide type for indices and byte sizes.

Partial blocks

When `n` is not divisible by the block size, extra threads must avoid global-memory access. In shared-memory algorithms, be careful: returning before a later `__syncthreads()` can make the remaining threads deadlock. Instead, inactive threads may need to load a neutral value, participate in the barrier, and skip only the final output.

Multi-dimensional blocks

The total thread count is the product of all dimensions. A block declared as `dim3(32, 32)` contains 1,024 threads, not 32. Device limits vary by GPU and should be queried rather than guessed when portability matters.

Warp divergence

Different branches within a warp may be serialized. Divergence is primarily a performance issue, but a divergent path containing a block barrier can also become a correctness issue if barrier participation is not uniform.

Kernel launch success does not mean execution success

The launch can be accepted and later fail due to an illegal address. Check both the launch and an appropriate completion point.

Asynchronous lifetime

Do not release or reuse memory while queued operations still depend on it. Stream ordering and synchronization determine when reuse is safe. Simple synchronous copies often provide an obvious completion point; pipelined programs require more deliberate lifetime management.

`cudaMemcpy` and overlapping regions

Treat `cudaMemcpy` like `memcpy`, not `memmove`: overlapping source and destination regions are not a safe general pattern.

cudaMemset is byte-oriented

`cudaMemset(d_values, 1, n * sizeof(int))` fills every byte with `0x01`; it does not produce integer value 1 in each element. Zero initialization works naturally because all-zero bytes represent zero for common CUDA numeric types.

Device-side kernel launches

A `__global__` function can be launched from device code only when using supported dynamic parallelism features and compilation/runtime requirements. For fundamentals interviews, the normal model is host launches kernel.

Grid-wide synchronization

There is no ordinary implicit grid-wide barrier inside a kernel. A kernel boundary is the simplest common global phase boundary. Cooperative groups provide specialized alternatives only when their constraints are satisfied.

Race despite a barrier

This remains wrong:

```
sharedCounter += 1;
__syncthreads();
```

The updates before the barrier race. Use an atomic operation or a structured reduction where each thread owns a distinct location before combining results.

11. How to Explain in Interview

CUDA is NVIDIA's parallel programming model. The CPU, or host, allocates device memory, copies inputs, and launches a `__global__` kernel as a grid of thread blocks. Inside the kernel, each thread combines `blockIdx`, `blockDim`, and `threadIdx` to find its data; `gridDim` helps form grid-wide strides. Threads in one block can share data and synchronize with `__syncthreads()`, but blocks are normally independent. I check allocation and copy APIs directly, then check both kernel launch and asynchronous execution errors. Finally, I copy required results back and release device memory with `cudaFree`.

For a whiteboard question, add the central formula:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) output[i] = operation(input[i]);
```

12. Quick Revision Notes

Key definitions

- **CUDA:** NVIDIA platform and programming model for general-purpose GPU computing.
- **Host:** CPU and its code/memory context.
- **Device:** GPU and its code/memory context.
- **Kernel:** `__global__` function executed by many device threads.
- **Grid:** All blocks created by one kernel launch.
- **Block:** Group of threads that can cooperate using shared memory and block barriers.
- **Thread:** One logical kernel execution instance.

Must-remember formulas

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
int blocks = (n + threadsPerBlock - 1) / threadsPerBlock;
int stride = blockDim.x * gridDim.x;
```

For 2D row-major data:

```
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
int i = y * width + x;
```

Important API sequence

```
cudaMalloc
-> cudaMemcpy HostToDevice
-> kernel launch
-> launch error check
-> synchronization/completion error check
-> cudaMemcpy DeviceToHost
-> cudaFree
```

Common comparisons

- `blockDim`: threads per block; `gridDim`: blocks per grid.
- `threadIdx`: local thread index; computed `i`: global data index.
- `__syncthreads()`: block barrier; `cudaDeviceSynchronize()`: host waits for device.
- `cudaGetLastError()`: launch/last-error state; synchronization: can reveal execution failure.
- Barrier: phase ordering; atomic: indivisible update.

Interview traps

- Kernel launches are normally asynchronous to the host.
- Extra threads are not automatically removed.
- Blocks have no guaranteed execution order.

- `__syncthreads()` is not grid-wide and must be reached uniformly.
- `cudaMalloc` memory is uninitialized.
- CUDA allocation/copy sizes are bytes.
- Successful kernel launch does not prove successful kernel execution.

13. Practice Tasks

Task 1: Vector addition

Write a kernel that computes `c[i] = a[i] + b[i]` for arbitrary `n`.

Check yourself: Use ceiling division, a global index, a bounds check, complete error handling, and cleanup.

Task 2: SAXPY

Implement `y[i] = alpha * x[i] + y[i]`. Test with an `n` that is not divisible by the block size, such as 1,003.

Interview extension: Explain whether the workload is compute-bound or memory-bandwidth-bound.

Task 3: Two-dimensional image kernel

Invert an 8-bit grayscale image with `output[i] = 255 - input[i]` using a 2D block and grid.

Check yourself: Convert `(x, y)` to a row-major linear index and guard both image dimensions.

Task 4: Grid-stride loop

Rewrite vector addition so each thread may process several elements. Launch a deliberately limited number of blocks and verify every element is covered once.

Task 5: Shared-memory neighbor operation

For each block, load input values into shared memory. Produce an output where each thread adds its value to the previous thread's value inside the block.

Check yourself: Place `__syncthreads()` after shared-memory loading and handle thread 0 separately.

Task 6: Find the barrier bug

Explain and fix this kernel fragment:

```
if (globalIndex < n) {
    shared[threadIdx.x] = input[globalIndex];
    __syncthreads();
    output[globalIndex] = shared[blockDim.x - 1 - threadIdx.x];
}
```

Hint: In a partial block, not every thread enters the branch. All threads must participate in the barrier; out-of-range threads should load a safe value and only valid threads should write output.

Task 7: Error-handling experiment

Launch a kernel with an intentionally invalid block size. Observe the result of `cudaGetLastError()`. Then create an out-of-bounds access and observe where an execution error is reported.

Use a disposable program and restore the correct code afterward; an illegal access can leave the CUDA context in an error state for that process.

Task 8: Resource cleanup

Write a function that allocates three device buffers. Make every failure path release allocations already completed, or use a small ownership wrapper if the exercise specifically allows modern C++ resource management.

Interview extension: Explain why terminating inside a general-purpose library's error macro may be inappropriate.

Task 9: Index tracing by hand

For `kernel<<<dim3(3, 2), dim3(4, 2)>>>`, calculate `(x, y)` for:

- `blockIdx = (0, 0), threadIdx = (0, 0)`
- `blockIdx = (2, 1), threadIdx = (3, 1)`
- Every thread in block `(1, 0)`

Task 10: CPU versus GPU reasoning

Given arrays of 64 elements, 1 million elements, and 1 billion elements, discuss when GPU offload may help. Include transfer cost, kernel launch overhead, data reuse, memory capacity, and available parallelism. Do not assume larger is always feasible.

Task 11: Reduction design

Design a block-level sum reduction using shared memory and `__syncthreads()`. Identify:

- Where each thread writes.
- Where barriers are necessary.
- How one result per block is produced.
- Why a second kernel or another mechanism is needed to combine block results.

Task 12: Minimal verification checklist

For every exercise:

1. Compute a CPU reference result.
 2. Run sizes `0, 1, blockSize - 1, blockSize, blockSize + 1`, and a larger irregular value.
 3. Check every CUDA API call.
 4. Compare GPU and CPU results, using a tolerance for floating-point calculations.
 5. Run a CUDA memory-checking tool when available to detect illegal accesses and races.
-

14. Final Cheat Sheet

Core definition

CUDA lets host code launch a device kernel over a hierarchy of grids, blocks, and threads to perform data-parallel work on NVIDIA GPUs.

Essential code pattern

```
__global__ void kernel(const float* input, float* output, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) output[i] = input[i] * 2.0f;
}

int threads = 256;
int blocks = (n + threads - 1) / threads;
kernel<<<blocks, threads>>>(d_input, d_output, n);
```

API essentials

API / syntax	Remember
<code>cudaMalloc(&ptr, bytes)</code>	Device allocation; uninitialized
<code>cudaMemcpy(dst, src, bytes, kind)</code>	Destination first; direction matters
<code>kernel<<<grid, block>>>(...)</code>	Normally asynchronous to host
<code>cudaGetLastError()</code>	Check launch/last error
<code>cudaDeviceSynchronize()</code>	Wait and reveal execution errors
<code>cudaFree(ptr)</code>	Release device allocation
<code>__syncthreads()</code>	Barrier only within one block

Function qualifiers

Qualifier	Executes on	Called from
<code>__host__</code>	CPU	CPU
<code>__device__</code>	GPU	GPU
<code>__global__</code>	GPU	Normally CPU launch

Most-asked questions

1. Explain grid, block, and thread.
2. Derive the global-index formula.
3. Why is a bounds check necessary?
4. Compare `__global__`, `__device__`, and `__host__`.

5. Explain all kernel launch configuration arguments.
6. Why are CUDA errors sometimes reported later?
7. What is the scope and restriction of `__syncthreads()`?
8. Compare a barrier with an atomic operation.
9. Why must blocks be independent?
10. Explain `cudaMalloc` → `cudaMemcpy` → kernel → `cudaFree`.

Five facts to never forget

1. Global index = `blockIdx.x * blockDim.x + threadIdx.x`
2. Grid size = `ceil(n / block size)`
3. Check `i < n`
4. `__syncthreads()` covers one block, not the grid
5. Check both launch errors and asynchronous execution errors

One-line interview answer

CUDA is a host-device programming model where the CPU launches a data-parallel kernel as a grid of independently scheduled thread blocks, with explicit memory management, indexing, synchronization, and error handling.